Course Outline: SPA Architecture & Bundling

Title: SPA Architecture & Bundling: Understanding How Modern Web Apps Are Built and Delivered **Learnpack:** + Arquitectura de Single Page Applications (SPA) y Empaquetado. **Prerequisite:** Memory Banks & Context Rules (Week 5, Day 14) **Target Audience:** Beginner developers who have built static pages and written TypeScript, but have never formally studied how web applications are compiled, delivered, or navigated **Total Lessons:** 19 **Estimated Total Length:** ~9,700 words **Format:** Conceptual-first with hands-on reinforcement (26% hands-on = 5 lessons)

Course Description

You've written HTML, styled components with Tailwind, and even used AI to scaffold a React project — but have you ever stopped to ask: what actually happens between writing your code and a user seeing it in their browser? This course answers that question.

Before diving into React and Next.js, you need a solid mental model of what a Single Page Application is, how it differs from a traditional website, and what "building" or "bundling" your code actually means. These concepts are the invisible infrastructure behind every modern frontend framework. Without understanding them, framework errors will feel mysterious and routing behavior will seem like magic.

This course builds that foundation. You'll learn what happens at build time versus runtime, why SPAs exist and what problems they solve, and how client-side routing strategies (hash-based and History API) create the illusion of page navigation without ever reloading the browser. By the end, you'll have the mental model you need to start working confidently with Next.js.

Course Philosophy

This course emphasizes:

- **Understanding before using:** Frameworks make more sense when you understand the problem they're solving
- **Evidence over assumption:** Inspect real browser behavior rather than taking concepts on faith
- **Path of least resistance:** Build from what students already know (HTML files, URLs, the browser) toward what's new (SPAs, bundlers, routing)

Course Statistics Summary

Section	Lessons	Estimated Words
00 - Welcome	1	~450
01 - From Source Code to Browser	3	~1,550
02 - Build Time vs. Runtime	3	~1,550
03 - Traditional Websites vs. SPAs	3	~1,550

Section	Lessons	Estimated Words
04 - Anatomy of a Single Page Application	3	~1,500
05 - Client-Side Routing Strategies	3	~1,550
06 - Knowledge Check	2	~1,150
07 - What's Next	1	~450
Total	**19**	**~9,750**

Section 00: Welcome

00.0 What You're About to Learn 🕮 (~450 words)

Learning Objectives:

- Understand the scope and purpose of this course
- Connect prior knowledge (HTML, CSS, TypeScript, Git) to what comes next (React, Next.js)
- Set expectations for what a "mental model" course looks like

Content Outline:

- Where you are in the 4Geeks curriculum and why this course exists here
- What a bundler is (teaser — not explained yet)
- What a Single Page Application is (teaser — not explained yet)
- Why understanding these concepts makes you a better AI-assisted developer
- What this course will NOT cover (React syntax, Next.js routing, component logic)

Transitions: This lesson orients students before any technical content begins. It connects their existing skill set — building static pages, writing TypeScript functions, using Git — to the bigger question of how a real web application actually reaches a user's browser.

Key Principles:

- Good developers understand the tools they use, not just how to type commands into them
- The browser is the final environment — everything you build is eventually interpreted there
- Mental models reduce confusion when things go wrong

Examples:

- Analogy: Writing a Word document vs. publishing a PDF — the source and the deliverable are different things
- Analogy: A recipe (source code) vs. a plated dish (built app) — users eat the dish, not the recipe

---

Section 01: From Source Code to Browser

01.0 What Does "Building" an App Actually Mean? 🕮 (~500 words)

Learning Objectives:

- Define what "building" or "compiling" a web application means
- Distinguish between source code (what developers write) and build output (what browsers receive)
- Identify the role of a bundler in the build process

Content Outline:

- The gap between developer files and browser-ready files
- What source code typically looks like: multiple files, modern syntax, imports, TypeScript
- What a browser can actually read: HTML, CSS, plain JavaScript — nothing else
- What a bundler does: takes your source files and transforms them into browser-compatible output
- Common bundlers by name only (Vite, Webpack, esbuild) — no configuration details
- The output of a build: typically a `/dist` or `/build` folder with a small number of optimized files

Transitions: This lesson establishes the fundamental idea that code must be transformed before it reaches users. The next lesson zooms into the specific mechanics of that transformation.

Key Principles:

- Browsers don't understand TypeScript, JSX, or ES module imports natively — your code must be translated
- A bundler is a build-time tool, not something that runs in the browser
- The build output is what gets deployed, not the source code

Examples:

- Show a simple TypeScript file (`const greet = (name: string): string => ...`) and contrast it with what the same code looks like after transpilation to plain JavaScript
- Show a project's `/src` folder (10+ files) next to its `/dist` folder (2-3 files) to illustrate consolidation

---

01.1 The Bundling Process: Packing Your Code for Delivery 🕮 (~500 words)

Learning Objectives:

- Describe the steps a bundler takes to transform source code
- Understand why bundling reduces file size and request count
- Recognize the terms "transpiling," "minifying," and "tree-shaking" without needing to configure them

Content Outline:

- Step 1 — Transpiling: Converting TypeScript or modern JS into browser-compatible JavaScript
- Step 2 — Bundling: Merging many files into fewer files to reduce HTTP requests
- Step 3 — Minifying: Removing whitespace, comments, and shortening variable names to reduce file size
- Step 4 — Tree-shaking: Removing code that is imported but never actually used
- What the final output looks like: `main.abc123.js`, `index.html`, `style.css`
- Why the file names often contain a hash (cache busting)

Transitions: Now that students understand what bundling produces, the next lesson makes the transformation tangible by having them inspect a real built app in the browser's developer tools.

Key Principles:

- Fewer HTTP requests = faster load times
- Smaller files = faster downloads
- Hash-based filenames ensure browsers don't serve stale cached versions

Examples:

- Before/after minification: a readable function vs. its compressed one-liner equivalent
- Analogy: Tree-shaking is like packing a suitcase — you only bring what you'll actually wear, not your entire wardrobe

---

01.2 Hands-On: Inspecting a Built App in the Browser 💻 (~550 words)

Learning Objectives:

- Navigate to the Sources and Network tabs in Chrome DevTools
- Identify bundled JavaScript files in a deployed web application
- Observe how a real SPA loads a single HTML file with JavaScript attached

Content Outline:

- Opening DevTools in Chrome (F12 or right-click → Inspect)
- The Network tab: filtering by JS and observing which files load
- The Sources tab: navigating the file tree of a loaded application
- Observing the root `index.html` of a real SPA — how minimal it is
- Comparing what the HTML says vs. what the user actually sees (JavaScript fills the gap)

Transitions: Students have now seen what the build output looks like from the outside. The next section asks a deeper question: when exactly does all of this happen?

Key Principles:

- The browser is your best debugging tool — learn to read what it tells you
- A minimal HTML file with a `<script>` tag is the foundation of most SPAs
- Evidence over assumption: always verify behavior in the browser before guessing

Hands-On Exercise: Open three real-world websites in Chrome (suggested: a news site, a web app like Notion or Trello, and a simple static page like a Wikipedia article).

1. Open DevTools → Network tab → filter by "JS"
2. Reload the page and observe: how many JS files loaded? What are their sizes?
3. Open DevTools → Sources tab → navigate the file tree
4. Right-click → View Page Source: find the root `index.html`

Questions to answer in writing:

- Which site loaded the most JavaScript? Which loaded the least?
- Could you read the JavaScript in the Sources tab, or was it minified?
- What was inside the `<body>` tag in the page source of each site?

Success Criteria: Student can locate the JS files loaded by a page, identify whether they are minified, and describe what the root HTML file contains (or doesn't contain) for each site.

Section 02: Build Time vs. Runtime

02.0 Build Time: What Happens Before the User Arrives 🕮 (~500 words)

Learning Objectives:

- Define "build time" and identify what work happens during it
- Explain why build-time errors are different from runtime errors
- Recognize TypeScript's role as a build-time tool

Content Outline:

- Definition: build time is everything that happens before the app is served to a user
- What runs at build time: the bundler, the TypeScript compiler, linters, tests
- Build-time errors: TypeScript type errors, missing imports, syntax errors — these prevent the app from building at all
- Why catching errors at build time is valuable (you find them before users do)
- The output of build time: static files ready to be served
- Connection to TypeScript: TS errors are build-time errors, not browser errors

Transitions: Build time prepares the app. Runtime is when the app actually runs. The next lesson covers what happens once the user opens their browser.

Key Principles:

- Build-time tools are invisible to the user — they run on your machine or a CI server
- A failed build means nothing gets deployed
- TypeScript was introduced earlier in this curriculum precisely because it catches problems before deployment

Examples:

- TypeScript error: calling a function with the wrong argument type → build fails → user never sees a broken page
- Analogy: A pre-flight checklist — pilots complete it before takeoff, not during the flight

---

02.1 Runtime: What Happens Inside the Browser 🕮 (~500 words)

Learning Objectives:

- Define "runtime" and identify what work happens during it
- Distinguish runtime errors from build-time errors
- Understand that the browser is the runtime environment for frontend code

Content Outline:

- Definition: runtime is everything that happens after the user opens the app in their browser
- The browser as a JavaScript runtime environment: it reads and executes your bundled JS
- What happens at runtime: rendering the UI, responding to user interactions, fetching data from APIs

- Runtime errors: `TypeError`, `ReferenceError`, null reference crashes — these happen while the user is using the app
- Why runtime errors are harder to prevent (they depend on real data, real user behavior, real network conditions)
- The relationship between runtime and the DevTools Console tab

Transitions: Both phases are necessary, but they serve different purposes and produce different kinds of errors. The next lesson builds a practical mental model that integrates both.

Key Principles:

- Build time is static and predictable; runtime is dynamic and unpredictable
- The Console tab in DevTools is your window into runtime behavior
- A successful build does not guarantee a working application — runtime is where real problems appear

Examples:

- Runtime error: an API returns `null` instead of an array, and your `.map()` call crashes
- Analogy: A printed recipe (build) vs. cooking the dish (runtime) — the recipe can be perfect but the dish can still burn

---

02.2 Thinking in Two Phases: A Developer's Mental Model 💻 (~550 words)

Learning Objectives:

- Correctly categorize errors as build-time or runtime
- Apply the two-phase mental model when debugging a broken application
- Identify which DevTools panel to open based on the type of problem

Content Outline:

- Recap: build time (before the user) vs. runtime (while the user is present)
- Decision framework: "Did the app fail to start, or did it crash while running?"
- Where to look for build-time problems: terminal output, compiler errors
- Where to look for runtime problems: DevTools Console, Network tab
- The middle ground: apps that build successfully but behave incorrectly (logic errors)
- Connecting this to TypeScript: TS moves certain runtime errors into build time

Transitions: Students now have a mental model for when code runs and what can go wrong at each stage. The next section uses this model to understand why Single Page Applications exist and what problem they were designed to solve.

Key Principles:

- Diagnose before you fix — knowing which phase produced the error narrows the search
- Not all problems are code problems: network failures, missing environment variables, and incorrect API responses are all runtime issues
- TypeScript is a build-time investment that pays off at runtime

Hands-On Exercise: You are given four error scenarios. For each one, answer:

- Is this a build-time or runtime error?
- What tool or tab would you open first to investigate?
- What is the likely cause?

Scenarios:

1. You run `npm run build` and the terminal shows: `Type 'string' is not assignable to type 'number'`
2. The app builds successfully. A user clicks a button and the browser Console shows: `Cannot read properties of undefined (reading 'map')`
3. You add a new import at the top of a file. The build fails with: `Module not found: Can't resolve './utils/helpers'`
4. The app loads fine on your machine. On a user's device, the page is blank and the Console shows: `Failed to fetch` on a network request

Success Criteria: Student correctly identifies 4/4 scenarios as build-time or runtime, names the correct tool to investigate, and provides a plausible explanation for each.

Section 03: Traditional Websites vs. SPAs

03.0 How Traditional Multi-Page Websites Work 🕮 (~500 words)

Learning Objectives:

- Describe the request-response cycle of a traditional multi-page website
- Identify what happens in the browser when a user navigates between pages
- Recognize the performance and UX tradeoffs of the traditional model

Content Outline:

- The traditional model: each URL corresponds to a separate HTML file on a server
- The full-page reload cycle: user clicks a link → browser sends a GET request → server returns a new HTML file → browser re-renders everything from scratch
- What the user experiences: a brief blank flash, a loading indicator, a fresh page render
- What gets re-downloaded on every navigation: HTML, CSS, JavaScript, fonts, images
- Where this model still makes sense: content-heavy sites, blogs, documentation, e-commerce with server-side rendering
- The connection to what students already know: the HTML files they built in Weeks 1–3 followed this model

Transitions: The traditional model works well for content sites but creates friction in application-like experiences. The next lesson explains what problems motivated the SPA approach.

Key Principles:

- Traditional websites are stateless from the browser's perspective — each page is a fresh start
- Full-page reloads are simple to reason about but expensive in terms of network and rendering work
- This model is not wrong — it's a deliberate tradeoff

Examples:

- Navigating Wikipedia: each article is a separate page, each click reloads the browser
- Contrast: clicking through a news site (full reload each time) vs. clicking through a music player (no reload, just updated content)

03.1 The Problems SPAs Were Born to Solve 🕮 (~500 words)

Learning Objectives:

- Identify the specific user experience problems that SPAs address
- Explain why application-like products (dashboards, social feeds, tools) need a different model
- Define a Single Page Application in precise terms

Content Outline:

- The UX problem: users expect desktop-application-like responsiveness from web products
- The performance problem: re-downloading CSS, JavaScript, and fonts on every navigation is wasteful
- The state problem: full-page reloads destroy in-memory application state (form data, scroll position, open modals)
- The definition: a Single Page Application is a web app that loads one HTML file once, then dynamically updates the DOM to simulate navigation — without ever reloading the page
- What makes it "single page": not that it has one screen, but that the browser never navigates away from the initial document
- Examples of real SPAs: Gmail, Figma, Notion, GitHub (partially), most React/Vue/Angular apps

Transitions: Now that students understand the problem SPAs solve, the next lesson makes the contrast concrete through direct comparison.

Key Principles:

- SPAs trade initial load complexity for smoother subsequent interactions
- The "single page" in SPA refers to the HTML document, not the number of views
- Application state lives in memory — losing it on reload is a serious UX problem

Examples:

- Gmail: composing an email, switching to inbox, switching back — your draft is still there
- Analogy: A traditional website is like a slide projector (new slide = new image). A SPA is like a whiteboard (you erase and redraw without replacing the board)

03.2 Comparing Side by Side: MPA vs. SPA 💻 (~550 words)

Learning Objectives:

- Identify whether a given website is an MPA or a SPA using DevTools
- Describe the tradeoffs of each approach in concrete terms
- Match the right architectural approach to a given product requirement

Content Outline:

- Recap: MPA (Multi-Page Application) vs. SPA — key differences

- Observable differences in the browser: full reload vs. no reload, Network tab activity, HTML file count in Sources
- When to use which: content-first sites (MPA) vs. application-first products (SPA)
- The hybrid middle ground (mention only): some frameworks serve HTML from the server but hydrate it client-side (this is what Next.js does — to be covered in the next learnpack)
- Tradeoffs table: initial load time, SEO, state management, development complexity

Transitions: Students can now identify and categorize web applications. The next section digs inside the SPA itself — how does it actually work once loaded?

Key Principles:

- Neither approach is universally superior — the right choice depends on the product
- SPAs are optimized for interaction; MPAs are optimized for content delivery and SEO
- The same application can use both strategies for different parts of the product

Hands-On Exercise: Open four websites and determine whether each is primarily an MPA or SPA. Use the Network tab (observe whether full-page reloads occur on navigation) and the Sources tab (look at the HTML structure).

Sites to investigate:

1. `wikipedia.org` (navigate between two articles)
2. `figma.com` (navigate between pages while logged out)
3. `docs.github.com` (navigate between documentation sections)
4. A site of the student's choice

For each site, document:

- Does clicking a link trigger a full page reload? (Yes/No — observe in Network tab)
- How many HTML files are visible in Sources?
- Is the root `index.html` minimal (a few lines) or full of content?
- Your conclusion: MPA, SPA, or hybrid?

Success Criteria: Student correctly identifies Wikipedia as MPA, Figma as SPA, and provides reasoned conclusions for the others based on observed browser behavior.

Section 04: Anatomy of a Single Page Application

04.0 One HTML File, Infinite Possibilities 🕮 (~500 words)

Learning Objectives:

- Describe what the single HTML file in a SPA actually contains
- Explain how JavaScript takes over rendering responsibility from the server
- Understand the role of the root DOM element (`<div id="root">`)

Content Outline:

- What the SPA's `index.html` looks like: minimal boilerplate, a `<div id="root">`, and a `<script>` tag

- How JavaScript renders the initial view: the script runs, queries the DOM for `#root`, and injects HTML dynamically
- The rendering pipeline: JS executes → reads application state → generates HTML string → injects into DOM
- Why the page source looks empty but the browser shows full content (JavaScript ran after the source was captured)
- The browser as a JavaScript runtime: the JS engine interprets your bundled code and manipulates the DOM
- Connection to bundling: the single `<script>` tag points to the bundled output file

Transitions: Students now understand the starting point of a SPA. The next lesson explains how it creates the illusion of multiple pages without ever leaving that one HTML document.

Key Principles:

- The HTML file is a container, not content — JavaScript provides the content
- "View Page Source" and "Inspect Element" show different things in a SPA (source shows the bare HTML; inspect shows the JavaScript-rendered DOM)
- The root element (`<div id="root">`) is the SPA's mounting point

Examples:

- Show a minimal SPA `index.html`: `<!DOCTYPE html><html><body><div id="root"></div><script src="main.js"></script></body></html>`
- Contrast: a traditional HTML page with 200 lines of content vs. a SPA's index.html with 12 lines

---

04.1 The Illusion of Navigation: How SPAs Fake Page Changes 🕮 (~500 words)

Learning Objectives:

- Explain how a SPA updates visible content without reloading the browser
- Describe the two strategies SPAs use to manage "which view to show": hash routing and History API routing
- Connect URL changes to view changes in a SPA

Content Outline:

- The problem: users expect the URL to reflect what they're looking at (bookmarking, sharing, back button)
- The solution: a SPA can change the URL without triggering a browser reload
- Strategy 1 — Hash routing: the URL changes after `#` (e.g., `example.com/#/about`) — browsers ignore the hash fragment when making server requests
- Strategy 2 — History API: the URL changes completely (e.g., `example.com/about`) using the browser's `history.pushState()` — no server request is triggered
- The router's job: read the current URL, match it to a view definition, render the correct component
- What happens when users press the browser Back button in a SPA

Transitions: This lesson introduces the two routing strategies at a conceptual level. The next section explores each in detail with hands-on observation.

Key Principles:

- URL changes in a SPA are cosmetic from the browser's perspective — no new document is fetched
- The router is just an if/else: "if the URL says `/about`, show the About view"
- The Back button works because routing history is tracked in memory (or via the History API)

Examples:

- Analogy: A SPA's router is like a TV remote — pressing a channel button changes what you see on the same screen, it doesn't replace the TV
- Show a URL changing from `app.com/#/home` to `app.com/#/settings` — same page, different hash

---

04.2 Anti-Patterns: Where SPA Thinking Goes Wrong 🕮 (~500 words)

Learning Objectives:

- Identify common mistakes developers make when building or working with SPAs
- Explain why each anti-pattern causes problems
- Recognize the correct approach in contrast to each anti-pattern

Content Outline:

Anti-Pattern 1: The Hard Refresh Trap Using a standard `<a href="/about">` anchor tag instead of a SPA router's link component. Result: the browser sends a real server request for `/about`, which either returns a 404 or re-downloads the entire app.

- Why it's common: `<a>` tags are what developers learn first
- Why it breaks SPAs: it exits the SPA model entirely
- Correct approach: always use the router's navigation mechanism (in React Router: `<Link to="/about">`)

Anti-Pattern 2: Treating the SPA Like a Multi-Page App Structuring a SPA with separate HTML files for each "page." This defeats the entire purpose of the SPA architecture.

- Why it's common: developers carry habits from traditional web development
- Correct approach: one HTML file, router-driven view switching

Anti-Pattern 3: Ignoring Build Errors Treating build-time failures as warnings or trying to test the app before the build succeeds.

- Why it's common: TypeScript and bundler errors can seem verbose or unclear
- Correct approach: fix the build before running anything — a broken build means the app doesn't exist yet

Transitions: Students now understand the full anatomy of a SPA and its failure modes. The next section goes deeper on the two routing strategies, with hands-on browser observation.

Key Principles:

- Anti-patterns in SPA development usually come from applying MPA habits in the wrong context
- Understanding the architecture makes anti-patterns obvious — you can see why they break the model

- Build errors are not optional — they are the compiler telling you the app cannot exist in its current state

---

Section 05: Client-Side Routing Strategies

05.0 Hash-Based Routing: The # Symbol Explained 🕮 (~500 words)

Learning Objectives:

- Explain the HTTP behavior of URL hash fragments
- Describe how hash-based routing enables navigation without server requests
- Identify the tradeoffs of hash routing (simplicity vs. limitations)

Content Outline:

- The `#` character in a URL: originally designed for in-page anchors (e.g., `page.html#section-2`)
- The key behavior: browsers do NOT send the fragment to the server — requests stop at the `?` or the path, never the `#`
- How SPAs exploit this: changing `#/home` to `#/about` triggers no server request, but the JS can read `window.location.hash` and update the view
- Implementing hash routing: listen to the `hashchange` event, parse the hash, render the matching view
- Tradeoffs: simple to implement, works without server configuration, but URLs look less clean (`/#/about` instead of `/about`)
- Where you still see hash routing: older SPAs, documentation sites (like some versions of Vue docs)

Transitions: Hash routing is the simpler strategy. The History API approach achieves the same result with cleaner URLs but requires more coordination between the frontend and the server.

Key Principles:

- The `#` fragment is a client-side contract — the server never sees it
- Hash routing requires zero server configuration, which makes it easier to deploy
- The `hashchange` event is the original mechanism for client-side routing

Examples:

- URL `https://myapp.com/#/dashboard` — the server receives a request for `/`, not `/dashboard`
- Show a simple hash router in pseudo-code: `window.addEventListener('hashchange', () => { render(window.location.hash) })`

05.1 Browser History API: The Modern Approach 🕮 (~500 words)

Learning Objectives:

- Explain what the History API does and why it was created
- Describe `history.pushState()` and how it changes the URL without reloading
- Identify the server configuration requirement that History API routing introduces

Content Outline:

- The History API: a browser feature that allows JavaScript to programmatically manipulate the browser's navigation history
- `history.pushState(state, title, url)`: changes the URL in the address bar without triggering a server request
- `window.onpopstate`: the event fired when the user presses Back or Forward
- How modern SPA routers use this: intercept link clicks → call `pushState` to update the URL → render the matching view
- The server-side catch: if a user bookmarks `myapp.com/about` and returns later, the browser requests `/about` directly from the server — which must return `index.html` (not a 404)
- The solution: configure the server to return `index.html` for all routes (a "catch-all" or "fallback" configuration)

Transitions: History API routing produces clean URLs and is the standard in modern frameworks. The final hands-on lesson in this section makes both routing strategies observable in a real browser.

Key Principles:

- `pushState` is not navigation — it's a URL display trick that triggers no server round-trip
- The server must be configured to support History API routing or direct URL access will fail
- All major SPA frameworks (React Router, Vue Router, Angular Router) use History API by default

Examples:

- Analogy: `pushState` is like changing the label on a filing cabinet drawer without moving any of the files inside
- The 404 problem: deploying to a static host without fallback configuration → works on navigation from home, breaks on refresh or direct URL access

---

05.2 Hands-On: Observing Routing Behavior in the Browser 💻 (~550 words)

Learning Objectives:

- Observe hash routing and History API routing in action using DevTools
- Confirm that client-side navigation triggers no server request
- Distinguish the two routing strategies by their URL format

Content Outline:

- Setting up: what to look for in the Network tab during SPA navigation
- Observable markers of hash routing: `#` in the URL, no new network request on navigation
- Observable markers of History API routing: clean URL change, no full-page reload indicator, no new document request in Network tab
- The Back button test: navigating forward and backward in a SPA to confirm history is preserved
- The direct URL access test: copying a deep link and pasting it into a new tab (does it work?)

Transitions: Students can now identify, describe, and observe both routing strategies in real applications. The final section tests this understanding before moving to the full course assessment.

Key Principles:

- No network request during navigation = client-side routing confirmed
- A full-page reload indicator (spinning tab favicon) means the browser navigated away from the SPA
- Direct URL access is the stress test for routing configuration

Hands-On Exercise: Investigate three real-world SPAs using the Network tab. For each one:

1. Open DevTools → Network tab → check "Preserve log"
2. Load the site's homepage
3. Navigate to a different section (click a link in the nav)
4. Observe: did a new document request appear in the Network tab?
5. Look at the URL: does it contain `#` or is it a clean path?
6. Press the browser's Back button: did the URL change? Did the page reload?

Suggested sites:

- `hashrouter.example.4geeks.com` (instructor provides a hash-routing demo)
- `trello.com` (navigate between boards — History API)
- A documentation site of the student's choice

For each site, document:

- Routing strategy (hash or History API)
- Evidence used to reach that conclusion
- What happened when you pressed Back

Success Criteria: Student correctly identifies the routing strategy for each site, provides Network tab evidence, and accurately describes the Back button behavior.

Section 06: Knowledge Check

06.0 Putting It Together: Mapping a Real App's Architecture 💻 (~550 words)

Learning Objectives:

- Apply all concepts from the course to analyze a real application
- Produce a written architectural description of a SPA using correct terminology
- Connect bundling, build time, runtime, and routing into a unified narrative

Content Outline:

- Review: the full lifecycle of a SPA from source code to user interaction
- The analysis task: pick a real application and describe its architecture
- What a good architectural description includes: build process, HTML structure, routing strategy, runtime behavior
- How to gather evidence: DevTools (Network, Sources, Console), page source, URL observation
- Writing with precision: using correct terms (bundler, build output, History API, root element, client-side routing)

Transitions: This practical lesson prepares students for the assessment by asking them to synthesize the course content into a coherent analysis. The assessment follows immediately.

Key Principles:

- Good developers can look at a running application and infer its architecture
- Evidence-based analysis is more valuable than guessing
- Writing a clear technical description is a skill — precision matters

Hands-On Exercise: Choose any web application you use regularly (a tool, a social platform, a dashboard). Using only your browser and DevTools:

Write a 200–300 word architectural analysis that answers:

1. Is this a SPA or an MPA? What evidence supports this?
2. Is the root `index.html` minimal or content-rich?
3. What routing strategy does it appear to use? How can you tell?
4. Approximately how many JS files loaded on the initial page load?
5. Did you encounter any build artifacts (hashed filenames, minified code)?
6. Describe what happens at build time vs. runtime for this application (inferred from evidence)

Success Criteria: Student produces a structured, evidence-backed analysis using correct course terminology. Conclusions align with the observed browser behavior. No claims are made without supporting evidence from DevTools.

06.1 SPA & Bundling Assessment 🧐 (~650 words)

Learning Objectives:

- Demonstrate understanding of the build process and the role of a bundler
- Accurately distinguish build-time from runtime behavior and errors
- Correctly explain how SPA routing creates the illusion of navigation

Question Format: Scenario-based (5 questions)

Question 1

A developer finishes building a React application and runs `npm run build`. The terminal shows:

```
TypeScript error: Argument of type 'string' is not assignable to parameter of type
'number'. (line 42)
Build failed with 1 error.
```

The developer says: "The app worked fine in development. I'll just deploy the development version for now."

- A) This is reasonable — build errors are usually minor style issues that don't affect functionality
- B) This is a problem — the build failed, meaning no deployable output was produced
- C) This is fine — TypeScript errors only matter if you want type safety, not for the app to work
- D) This is unnecessary — the browser can run TypeScript directly without building

Correct answer: B **Explanation:** A build failure means no output files were produced. There is nothing to deploy. TypeScript errors at build time exist precisely to prevent broken code from reaching users. Deploying the development version is not a valid workaround — development mode is not optimized for production and exposes source code.

Question 2

A user is using a project management SPA. She opens the app, navigates to `/projects/42/tasks`, and then refreshes the page. She receives a 404 error. The application works perfectly when she navigates using the app's links.

What is the most likely cause?

- A) The app's JavaScript has a bug that only appears on refresh
- B) The server is not configured to return `index.html` for all routes, so direct URL access fails
- C) Hash routing was used incorrectly — the URL should contain `#` before `/projects`
- D) The bundler failed to include the `/projects` route in the build output

Correct answer: B **Explanation:** This is the classic History API routing problem. Navigation within the SPA works because the router intercepts clicks and uses `pushState` — no server request is made. But a browser refresh sends the full URL path `/projects/42/tasks` directly to the server, which has no file at that path. The server must be configured with a catch-all that returns `index.html` for all routes.

Question 3

A developer is building a simple SPA and wants to use hash-based routing. She observes that navigating between views does not appear in the Network tab. Her colleague says this is a bug.

- A) The colleague is correct — every navigation should produce a network request
- B) The colleague is incorrect — hash routing intentionally produces no server request; this is expected behavior
- C) The colleague is correct — even hash routing requires a server response for each route change
- D) The colleague is incorrect — but only if the app is in development mode; in production it would show requests

Correct answer: B **Explanation:** Hash routing exploits the browser's behavior with URL fragments: the part of the URL after `#` is never sent to the server. When the hash changes, the browser fires a `hashchange` event and the JS router updates the view — no network request occurs. This is not a bug; it is the core mechanism that makes hash routing work.

Question 4

A developer opens a website's page source and sees only 8 lines of HTML: a `<!DOCTYPE>`, a `<head>` with metadata, and a `<body>` containing a single `<div id="app">` and a `<script src="main.a3f91b.js">`. But when she opens the page in the browser, she sees a full, complex application with a navigation bar, a sidebar, and a data table.

What best explains this?

- A) The server is injecting content before the page renders, which doesn't appear in source
- B) The page source is outdated — the browser caches an older, fuller version of the HTML
- C) The JavaScript file runs at runtime and dynamically renders all visible content into `#app`
- D) This is a static site — all content is embedded in the JS file as HTML strings and shown on load

Correct answer: C **Explanation:** This describes the standard SPA rendering model. The HTML file is a minimal container. At runtime, the browser downloads and executes `main.a3f91b.js`, which renders the full UI by dynamically writing into the `#app` element. Page source shows what was delivered by the server; Inspect Element shows what JavaScript produced after execution.

Question 5

Which of the following correctly describes the difference between build time and runtime?

- A) Build time is when the browser renders the UI; runtime is when developers write the code
- B) Build time happens on the developer's machine (or CI server) before deployment; runtime happens in the user's browser after the app loads
- C) Build time is the same as runtime for SPAs — everything happens inside the browser
- D) Runtime errors are easier to catch than build-time errors because users report them directly

Correct answer: B **Explanation:** Build time is the phase during which the bundler, TypeScript compiler, and other tools transform source code into deployable output. This happens before any user touches the app. Runtime is everything that happens after a user loads the app — JavaScript executes, the UI renders, API calls are made, and errors can occur based on real data and real user behavior.

Evaluation Criteria:

- 5/5 correct: Strong conceptual foundation — ready for Next.js
- 4/5 correct: Solid understanding with one area to revisit
- 3/5 correct: Review sections on routing strategies and build/runtime distinction before proceeding
- Below 3/5: Re-read Sections 02 and 05 before continuing

Section 07: What's Next

07.0 You've Got the Foundation — Now Let's Build (~450 words)

Content Outline:

What you accomplished in this course: You started with a question most developers never stop to ask: what actually happens between writing code and a user seeing it in a browser? Over the course of 18 lessons, you built a complete mental model of that process.

You now know that source code must be transformed by a bundler before browsers can read it, that this transformation happens at build time — before any user arrives — and that a completely different phase called runtime is where real behavior, real errors, and real user interactions occur.

You explored why Single Page Applications exist: not as a trend, but as a deliberate solution to specific UX and performance problems that traditional multi-page websites couldn't solve. You dissected the anatomy of a

SPA — the minimal HTML file, the root element, the JavaScript that does all the actual rendering. And you learned that the "single page" part isn't a limitation; it's the entire point.

You also made the invisible visible. Using DevTools, you observed hash routing and History API routing in real applications, confirmed that client-side navigation produces no server requests, and learned to read the Network and Sources tabs as diagnostic tools rather than curiosities.

How this connects to what comes next: The next learnpack introduces Next.js — a framework built on React that runs on both the server and the browser. Everything you learned here applies directly:

- When Next.js builds your app, that's the bundler you just learned about
- When you configure routes in Next.js, you're using the History API under the hood
- When you see a minimal HTML file in a Next.js project, you'll recognize it immediately
- When Next.js talks about "client-side" vs. "server-side" rendering, you now know exactly what those phases mean

Resources for continued exploration:

- MDN Web Docs — History API: `developer.mozilla.org/en-US/docs/Web/API/History_API`
- MDN Web Docs — hashchange event: `developer.mozilla.org/en-US/docs/Web/API/Window/hashchange_event`
- Vite documentation (the bundler you'll use in Next.js projects): `vitejs.dev`
- Chrome DevTools documentation: `developer.chrome.com/docs/devtools`

A note before you move on: The concepts in this course are the kind that make more sense the second time you encounter them — once you're actually inside a React or Next.js project and you see the folder structure, the build output, and the router in action. Come back to these ideas whenever something in the next learnpack feels unclear. The architecture is always the same; the syntax is just new clothes on a familiar body.

You're ready. Let's build.

End of Course Outline: SPA Architecture & Bundling